

LAB EXERCISE 2: BASIC C AND INPUT/OUTPUT

Course Code: CCC1123 – Fundamental of C and Basic Input Output

Duration: 1 hour 30 minutes

Week: 2

Learning Outcomes

At the end of this lab, students should be able to:

1. Declare and use variables and constants correctly.
2. Apply basic data types and format specifiers.
3. Use `printf( )` and `scanf( )` or `fgets( )` for user input and formatted output.

TASK 1: Variable and Constant Declaration (20 minutes)

Objective: Understand how to declare variables and constants in C.

Instructions:

1. Open a new C source file.
2. Write a simple program that:
 - Declares variables of type `int`, `float`, `char`, and `double`.
 - Declares a constant using the `const` keyword.
 - Assigns values to each variable.
 - Displays them using `printf()` with correct format specifiers.

Example Output:

```
Integer value      : 25
Floating value     : 3.50
Character value    : A
Double value       : 1234.567890
Constant value     : 3.14
```

Hints: Use `%d`, `%f`, `%c`, `%lf` as format specifiers. Use `const` to declare a fixed value.

TASK 2: Understanding Data Types and Type Conversion (30 minutes)

Objective: Apply different data types and understand implicit and explicit conversions.

Instructions:

1. Create a new program file.
2. Declare the following variables:

```
int a = 5, b = 2;  
float result;
```
3. Compute a / b and display the result as an integer division.

What is the result and explain?.

4. Then perform type conversion to get a floating-point result using:

```
result = (float)a / b;
```
5. Display both results for comparison.

Expected Output:

Integer division result: 2

Floating-point division result: 2.50

TASK 3: User Input, Arithmetic Operation and Output Display (40 minutes)

Objective: Use input/output functions (`scanf`, `fgets`) to interact with users, perform arithmetic operations, and display results clearly.

Instructions:

1. Create a new program file.
2. Write a C program that performs the following tasks:
Step 1: Display a program title using `printf( )` statement:

==== Student Total Marks Calculation System ====

- Step 2: Declare the following variables:

```
char studentName[50];  
float quiz, assignment, project, midTerm, finalExam;  
float carryMark, totalMark;
```

- Step 3: Prompt the user to enter the following data:

- Student full name (use `fgets()` to accept names with spaces)
- Quiz mark (10%)
- Assignment mark (10%)
- Project mark (20%)
- Mid-term mark (20%)

- Final exam mark (40%)

Step 4: Calculate the carry mark (60%) and total mark (100%):

```
carryMark = quiz + assignment + project + midTerm;  
totalMark = carryMark + finalExam;
```

Step 5: Display the results clearly with two decimal places using printf().

Sample Output:

```
===== Student Total Marks Calculation System =====  
Enter student full name      : Baharudin Osman  
Enter quiz mark (10%)       : 8.5  
Enter assignment mark (10%) : 9.0  
Enter project mark (20%)    : 15.0  
Enter mid-term mark (20%)   : 18.0  
Enter final exam mark (40%) : 35.5  
  
--- Student Total Marks Summary ---  
Student Name      : Baharudin Osman  
Carry Mark (60%)  : 50.50  
Final Exam (40%)  : 35.50  
Total Mark (100%) : 86.00
```

TASK 4: Display Floating Point Values in Exponential Form (extra time)

Objective: Use the %e and %E format specifiers to display floating-point values in scientific (exponential) notation.

Instructions:

1. Create a new program file.

2. Write a C program that:

- Declares num1 and num2 with the following floating-point values (float, double).

```
float num1 = 12345.6789;
```

```
double num2 = 0.000012;
```

- Assigns large and small decimal values.

- Displays the values using:

 - %f (normal),

 - %e (lowercase exponential),

 - %E (uppercase exponential).

3. Include comments that explain the difference between %f, %e, and %E.

Hint:

- %f → Displays in decimal format (fixed point).

- %e → Displays in scientific notation (e.g., 1.23e+03).

- %E → Same as %e but uses uppercase 'E'.

- You can control decimal precision using %.2e, %.4E, etc.

Sample Output:

===== Display Floating-Point Values in Exponential Form =====

Number 1 in normal format : 12345.678900

Number 1 in exponential(%e) : 1.234568e+04

Number 1 in exponential(%E) : 1.234568E+04

Number 2 in normal format : 0.000012

Number 2 in exponential(%e) : 1.200000e-05

Number 2 in exponential(%E) : 1.200000E-05